

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	Academic Year:2025-2026
Course Coordinator Name		Venkataramana Veeramsetty	
Instructor(s) Name		Dr. V. Venkataramana (Co-ordinator)	
		Dr. T. Sampath Kumar	
		Dr. Pramoda Patro	
		Dr. Brij Kishor Tiwari	
		Dr.J.Ravichander	
		Dr. Mohammand Ali Shaik	
		Dr. Anirodh Kumar	
		Mr. S.Naresh Kumar	
		Dr. RAJESH VELPULA	
		Mr. Kundhan Kumar	
		Ms. Ch.Rajitha	
		Mr. M Prakash	
		Mr. B.Raju	
		Intern 1 (Dharma teja)	
		Intern 2 (Sai Prasad)	
		Intern 3 (Sowmya)	
		NS_2 (Mounika)	
Course Code	24CS002PC2 15	Course Title	AI Assisted Coding
Year/Sem	II/I	Regulation	R24
Date and Day of Assignment	Week10 - Thursday	Time(s)	
Duration	2 Hours	Applicable to Batches	
AssignmentNumber:19.4(Present assignment number)/24(Total number of assignments)			
	Question		Expec ted Time to compl ete
1	Lab 19 – Code Translation: Converting Between Programming Languages		Week 10 - Thursd ay

Lab Objectives:

- Understand how AI tools can assist in translating code between different programming languages.
- Learn to verify correctness and functionality after translation.
- Explore syntactic and semantic differences between languages (e.g., Python, Java, C++).
- Practice debugging and optimizing AI-translated code.

Task 1: Translate a Simple Program (Python → JavaScript)**• Instructions:**

- Write a Python function `print_numbers()` that prints the first 10 natural numbers using a loop.
- Translate the function into JavaScript as a reusable function `printNumbers()`.
- Call the function in both languages to display results.

• Expected Output:

- 1
- 2
- 3
- ...10

PROMPT :

- **Generate** a Python function `print_numbers()` that prints the first 10 natural numbers using a loop.
- Translate the function into JavaScript as a reusable function `printNumbers()`.
- Call the function in both languages to display results.

OUTPUT:

```
def print_numbers_python():
    """Prints the first 10 natural numbers using a loop in Python."""
    for i in range(1, 11):
        print(i)
```

```
print_numbers_python()
```

```
1
2
3
4
5
6
7
8
9
10
```

```
from IPython.display import display, Javascript
```

```
# Call the Python function
print("Python output:")
print_numbers_python()
```

```
# Call the JavaScript function
print("\nJavaScript output:")
js_code = """
printNumbersJavaScript();
"""
display(Javascript(js_code))
```

```
Python output:
1
2
3
4
5
6
7
8
9
10
```

```
JavaScript output:
```

OBSERVATION:

The Python function executed successfully and printed the numbers 1 through 10. The JavaScript function, although defined, was not directly executable in the Python cell, resulting in a `SyntaxError`. The call to the Python function from the final Python cell was successful, but the call to the JavaScript function using `IPython.display.Javascript` did not produce output visible in the standard output stream.

Task 2: Convert Conditional Statements (Java → Python)

• Instructions:

- Write a Java method `checkNumber(int num)` that checks if a number is positive, negative, or zero.
- Translate the method into a Python function `check_number(num)`.

- Call the function/method with different inputs and compare outputs.
- **Expected Output:**
 - Input: -5 → Output: The number is negative
 - Input: 0 → Output: The number is zero
 - Input: 7 → Output: The number is positive

CODE :

```
# This task requires generating Java code, which cannot be directly executed in a Python notebook.
# The following is a representation of the Java code based on the requirements.
```

```
'''
public class NumberChecker {
    public static void checkNumber(int num) {
        if (num > 0) {
            System.out.println("The number is positive");
        } else if (num < 0) {
            System.out.println("The number is negative");
        } else {
            System.out.println("The number is zero");
        }
    }
}
'''
```

```
# Note: The above is Java code and will not execute here.

'\npublic class NumberChecker {\n    public static void checkNumber(int num) {\n        if (num > 0) {\n            System.out.println\n        }\n        } else if (num < 0) {\n            System.out.println("The number is negative");\n        } else\n        {\n            System.out.println("The number is zero");\n        }\n    }\n}'
```

```
def check_number(num):
    """Checks if a number is positive, negative, or zero and prints the result."""
    if num > 0:
        print("The number is positive")
    elif num < 0:
        print("The number is negative")
    else:
        print("The number is zero")
```

```
print("Input: -5 \u2192 Output:", end=" ")
check_number(-5)
```

```
print("\nInput: 0 \u2192 Output:", end=" ")
check_number(0)
```

```
print("\nInput: 7 \u2192 Output:", end=" ")
check_number(7)
```

```
print("\nNote: Direct execution and comparison with the Java code is not feasible in this environment. The comparison is based on the expected output format.")
```

```
Input: -5 → Output: The number is negative
```

```
Input: 0 → Output: The number is zero
```

```
Input: 7 → Output: The number is positive
```

```
Note: Direct execution and comparison with the Java code is not feasible in this environment. The comparison is based on the expected output format.
```

OBSERVATION:

The Python function `check_number` successfully classified the input numbers (-5, 0, and 7) as negative, zero, and positive, respectively, matching the expected output format. While the Java

code was provided as a string due to the environment limitations, its logic is functionally equivalent to the Python code for this task. The comparison is based on the expected behavior of both implementations.

Task 3: Translate Recursive Function (Python → C++)

- **Instructions:**

- Write a Python function `factorial(n)` that calculates factorial of a number using recursion.
- Translate the same into a C++ function `int factorial(int n)`.
- Call the function in both languages with inputs 5 and 0.

- **Expected Output:**

- **Input: 5 → Output: Factorial = 120**
- **Input: 0 → Output: Factorial = 1**

CODE:

```
def factorial(n):  
    """Calculates the factorial of a non-negative integer using recursion."""  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n - 1)
```

```
# This task requires generating C++ code, which cannot be directly executed in a Python notebook.  
# The following is a representation of the C++ code based on the requirements.
```

```
cpp_code = """  
#include <iostream>  
  
int factorial(int n) {  
    if (n == 0) {  
        return 1;  
    } else {  
        return n * factorial(n - 1);  
    }  
}  
"""
```

```
print(cpp_code)
```

```
#include <iostream>  
  
int factorial(int n) {  
    if (n == 0) {  
        return 1;  
    } else {  
        return n * factorial(n - 1);  
    }  
}
```

```
print("Input: 5 \u2192 Output: Factorial =", factorial(5))
print("Input: 0 \u2192 Output: Factorial =", factorial(0))

print("\nNote: Direct execution and comparison with the C++ code is not feasible in this environment. The comparison is based on the e

Input: 5 → Output: Factorial = 120
Input: 0 → Output: Factorial = 1

Note: Direct execution and comparison with the C++ code is not feasible in this environment. The comparison is based on the expected lo
```

OBSERVATION:

The Python `factorial` function successfully calculated the factorial for inputs 5 and 0, producing the expected outputs of 120 and 1, respectively. The C++ code for the `factorial` function, although provided as a string due to environment limitations, demonstrates the same recursive logic as the Python function, with a base case for $n=0$ and a recursive step for $n>0$. The comparison is based on the equivalent logic implemented in both languages.

Task 4: Data Structures with Functions (JavaScript → Python)

- **Instructions:**

- Write a JavaScript function `printStudents(students)` that takes an array of student names and prints each name.
- Translate it into a Python function `print_students(students)` using a list.
- Test both functions with sample student names.

- **Expected Output:**

- Student List:
- Alice
- Bob
- Charlie

CODE:

```
# This task requires generating JavaScript code, which cannot be directly executed in a Python notebook.  
# The following is a representation of the JavaScript code based on the requirements.
```

```
js_code = """  
function printStudents(students) {  
    /** Prints each student name in the given array. */  
    for (let i = 0; i < students.length; i++) {  
        console.log(students[i]);  
    }  
}  
"""  
  
print(js_code)
```

```
function printStudents(students) {  
    /** Prints each student name in the given array. */  
    for (let i = 0; i < students.length; i++) {  
        console.log(students[i]);  
    }  
}
```

```
def print_students(students):  
    """Prints each student name in the given list."""  
    for student in students:  
        print(student)
```

```
from IPython.display import display, Javascript  
  
# 1. Define a Python list named sample_students  
sample_students = ["Alice", "Bob", "Charlie"]  
  
# 2. Call the Python function print_students  
print("Python Output:")  
print_students(sample_students)  
  
# 3. Define a JavaScript array as a string  
js_students_array = '["Alice", "Bob", "Charlie"]'  
  
# 4. Construct a JavaScript code string  
js_code_execution = f"""  
console.log("JavaScript Output:");  
printStudents({js_students_array});  
"""  
  
# 5. Use IPython.display.display and IPython.display.Javascript to execute the JavaScript code  
display(Javascript(js_code_execution))
```

```
Python Output:  
Alice  
Bob  
Charlie
```

OBSERVATION:

The Python function `print_students` successfully iterated through the list of student names and printed each one on a new line, matching the core requirement. The JavaScript function `printStudents`, while defined, was executed within the browser environment via `IPython.display.Javascript`. To fully confirm its output, one would need to check the browser's developer console or the configured output area, as

console.log output from JavaScript in this context typically appears there rather than in the standard notebook output. Both functions demonstrate the same fundamental logic for iterating and printing elements from a list/array.

Task 5: Class & Object Translation (Python → Java)

- **Instructions:**

1. Write a **Python class** Car with attributes: brand, model, year.
2. Add a **method** display_details() that prints car details.
3. Translate the same into a **Java class** Car with attributes and a **method** displayDetails().
4. Create an object in both languages and call the method.

- **Expected Output:**

- Car Details:
- Brand: Toyota
- Model: Corolla
- Year: 2020

CODE:

```
class Car:
    """Represents a car with brand, model, and year."""

    def __init__(self, brand, model, year):
        """Initializes a Car object."""
        self.brand = brand
        self.model = model
        self.year = year

    def display_details(self):
        """Prints the details of the car."""
        print(f"Car Details: Brand: {self.brand} Model: {self.model} Year: {self.year}")
```



```

java_code = """
public class Car {
    String brand;
    String model;
    int year;

    public Car(String brand, String model, int year) {
        this.brand = brand;
        this.model = model;
        this.year = year;
    }

    public void displayDetails() {
        System.out.println("Car Details: Brand: " + brand + " Model: " + model + " Year: " + year);
    }
}
"""
print(java_code)

```

```

public class Car {
    String brand;
    String model;
    int year;

    public Car(String brand, String model, int year) {
        this.brand = brand;
        this.model = model;
        this.year = year;
    }

    public void displayDetails() {
        System.out.println("Car Details: Brand: " + brand + " Model: " + model + " Year: " + year);
    }
}

```

```

# 1. Create an instance of the Car class in Python
my_car = Car(brand="Toyota", model="Corolla", year=2020)

# 2. Call the display_details() method on the created Python Car object
my_car.display_details()

# 3. Print a note explaining the limitation of comparison with Java
print("\nNote: Direct execution and comparison with the Java code is not feasible in this environment. The comparison is based on the

```

Car Details: Brand: Toyota Model: Corolla Year: 2020

Note: Direct execution and comparison with the Java code is not feasible in this environment. The comparison is based on the expected 1

OBSERVATION:

The Python Car class and its `display_details` method were successfully defined and executed, producing the expected output format with the provided car details. The Java Car class, presented as a string, shows a logically equivalent structure with attributes, a constructor, and a `displayDetails` method that would produce the same output format if compiled and run in a Java environment. The comparison confirms the similarity in object-oriented structure and method implementation between Python

and Java for this task.

✓ Deliverables (For All Tasks)

1. AI-generated prompts for code and test case generation.
2. At least 3 assert test cases for each task.
3. AI-generated initial code and execution screenshots.
4. Analysis of whether code passes all tests.
5. Improved final version with inline comments and explanation.
6. Compiled report (Word/PDF) with prompts, test cases, assertions, code, and output.

Top of Form